

Cours 05 — La maîtrise de l’historique

Module 5/8 — Formation Git & GitHub

Objectifs

À la fin de ce module, vous saurez : - réécrire l’historique **local** avec `commit --amend` et `rebase` (simple et interactif) ; - mettre un travail de côté avec `stash`, transplanter un commit précis avec `cherry-pick` ; - annuler proprement en distinguant `reset` (déplace la branche, **local**) de `revert` (nouveau commit, **public**) ; - rattraper une fausse manœuvre avec le filet de sécurité : `reflog`.

Théorie

La règle d’or : ne jamais réécrire l’histoire partagée

Ce module manipule l’historique — c’est puissant, et c’est là qu’on peut se faire mal. Une seule règle protège de tout :

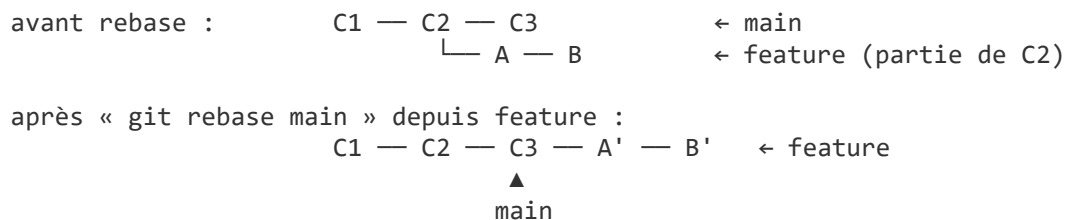
On ne réécrit jamais un historique déjà poussé et partagé.

Tant que vos commits sont **locaux** (ou seuls sur votre branche de PR personnelle que personne d’autre n’utilise), vous pouvez tout refaire : les fusionner, les reformuler, les supprimer. Dès qu’ils sont sur une branche **partagée** (`main`, une branche sur laquelle un collègue travaille), on n’ajoute plus que de **nouveaux** commits — et pour annuler, on utilise `revert`, jamais `reset` ni `rebase`.

Pourquoi ? Réécrire change les hash des commits. Si d’autres ont déjà les anciens, leurs dépôts et le vôtre racontent deux histoires incompatibles : la synchro suivante devient un cauchemar.

Rebase : rejouer plutôt que fusionner

`git rebase main` prend vos commits et les **rejoue par-dessus** la pointe de `main`. Là où `merge` crée un commit de fusion pour réconcilier deux histoires, `rebase` réécrit la vôtre pour qu’elle **reparte** du sommet de `main` : l’historique devient **linéaire**, sans embranchement.



A et B deviennent A' et B' : même contenu, **nouveaux hash** (ils ont un nouveau parent). C’est exactement pour ça que la règle d’or existe.

Rebase interactif : faire le ménage avant une PR

`git rebase -i HEAD~3` ouvre un éditeur listant vos 3 derniers commits, chacun précédé d’une action modifiable :

Action	Effet
pick	garder le commit tel quel (défaut)
reword	garder le commit mais reformuler son message
squash	fusionner ce commit dans le précédent (garde les deux messages)

Action	Effet
fixup	comme squash, mais jette le message de ce commit
drop	supprimer le commit

C'est l'outil pour transformer une série de « wip », « fix », « oups » en **un** commit propre, juste avant d'ouvrir la PR. Les relecteurs vous en remercieront.

Reset vs revert : deux façons d'annuler

C'est la distinction à ne pas rater du module.

Commande	Effet	Historique	Usage
<code>reset --soft HEAD~1</code>	Reculer la branche d'un commit, garde les modifs dans l'index	Réécrit	Refaire le dernier commit
<code>reset --hard HEAD~1</code>	Reculer la branche, détruit les modifs du répertoire de travail	Réécrit	Abandon local — avec prudence
<code>revert <hash></code>	Crée un nouveau commit qui défait l'ancien	Ajoute	Annuler sur une branche partagée

L'image : `reset` **remonte le temps** (il déplace l'étiquette de branche en arrière, comme si les commits suivants n'avaient jamais eu lieu) — donc réservé au local. `revert` **avance dans le temps pour corriger** (il ajoute un commit qui est l'inverse d'un précédent) — donc sûr même sur `main`, puisqu'il ne réécrit rien.

Stash : mettre de côté sans committer

`git stash` range vos modifications en cours dans une pile temporaire et **nettoie** le répertoire de travail. Utile quand un collègue vous demande d'urgence de regarder autre chose : vous ne voulez pas committer un travail à moitié fait, mais vous devez changer de branche. On remise (`stash`), on bascule, on revient, on ressort (`stash pop`).

Le filet de sécurité : reflog

`git reflog` journalise **tous** les déplacements de HEAD sur votre machine — y compris ceux qu'un `reset --hard` semble avoir effacés. Un commit « perdu » par une fausse manœuvre y reste accroché **~90 jours**. Tant que vous avez le reflog, presque rien n'est vraiment perdu.

Commandes commentées

Corriger le dernier commit — `git commit --amend`

```
git commit --amend -m "Message corrigé"    # remplace message ET/OU contenu du
dernier commit
```

Faute de frappe dans le message, ou fichier oublié dans le dernier commit ? `--amend` **remplace** le dernier commit au lieu d'en créer un nouveau (pour ajouter un fichier oublié : `git add le-fichier` d'abord, puis `git commit --amend`). Attention : `--amend` **réécrit** le commit (nouveau hash) — donc uniquement s'il n'a pas encore été poussé sur une branche partagée.

Mettre de côté — git stash

```
git stash                # remise le travail en cours, nettoie le
répertoire
git stash list           # lister la pile de remises (stash@{0},
stash@{1}...)
git stash pop            # ressort la dernière remise ET la retire de la
pile
git stash apply          # ressort SANS retirer de la pile (utile pour
ré-appliquer)
```

stash fonctionne en pile : pop ressort la plus récente. Astuce : nommez vos remises avec `git stash push -m "wip export CSV"` pour vous y retrouver quand la pile grossit.

Rebaser sur main à jour — git rebase

```
git switch feature/export-csv
git rebase main          # rejoue vos commits par-dessus la pointe de
main
# en cas de conflit : éditer le fichier, puis
git add fichier-resolu
git rebase --continue    # OU tout annuler proprement : git rebase --
abort
```

Avant d'ouvrir (ou de mettre à jour) une PR, on se rebase sur `main` pour intégrer ce qui a avancé et garder un historique droit. Un rebase peut lever plusieurs conflits **d'affilée** (un par commit rejoué) : on les résout un à un, `git rebase --continue` entre chaque. En cas de doute, `git rebase --abort` remet tout comme avant.

Nettoyer ses commits — git rebase -i

```
git rebase -i HEAD~4     # ouvre l'éditeur pour les 4 derniers commits
# remplacez « pick » par squash / fixup / reword / drop selon le besoin,
# sauvegardez et fermez : Git rejoue selon vos instructions
```

Le rebase interactif est le grand nettoyeur d'avant-PR. Exemple : quatre commits « wip », « wip », « fix », « ça marche » → mettez `pick` sur le premier et `fixup` sur les trois autres → il n'en reste **qu'un**, propre.

Transplanter un commit — git cherry-pick

```
git cherry-pick a3f9c12   # applique CE commit précis sur la branche
courante
```

`cherry-pick` copie **un** commit (identifié par son hash, lu dans `git log`) et le rejoue ici. Typique : un correctif fait sur une branche est aussi nécessaire sur une autre, sans vouloir fusionner toute la branche. On récolte juste la cerise.

Annuler — git revert et git reset

```
git revert a3f9c12        # crée un NOUVEAU commit qui défait a3f9c12 –
sûr sur main
git reset --soft HEAD~1   # défait le dernier commit, GARDE les modifs
indexées – local
git reset --hard HEAD~1   # défait le dernier commit et DÉTRUIT les modifs
– local, prudent
```

`revert` pour le partagé (il ajoute, ne réécrit rien). `reset` pour le local uniquement. `--soft` garde votre travail (pour recommitter autrement), `--hard` le jette (vérifiez `git status` **avant**, il n'y a pas de corbeille — juste le reflog).

Rattraper une erreur — git reflog

```
git reflog                                # journal de TOUS les déplacements de HEAD
# ex. : a3f9c12 HEAD@{2}: commit: travail que je croyais perdu
git reset --hard HEAD@{2}                # revenir précisément à cet état-là
```

Après un `reset --hard` malheureux, `reflog` retrouve l'état d'avant : chaque ligne est un point de retour (`HEAD@{n}`). On y revient avec `git reset --hard HEAD@{n}`. C'est votre machine à remonter le temps — apprenez à l'aimer avant d'en avoir besoin.

Exercices pratiques

Travaillez sur une branche de test pour manipuler sans risque.

1. **Amend.** Committez avec une faute dans le message, puis corrigez-la avec `git commit --amend`.
2. **Rebase interactif.** Faites quatre petits commits (« wip », « fix », ...), puis fusionnez-les en **un seul** commit propre avec `git rebase -i HEAD~4`.
3. **Reset vs revert.** Sur une branche de test, annulez un commit avec `reset --hard`, puis un autre avec `revert`. Comparez les deux historiques avec `git log --oneline`.
4. **Sauvetage.** Faites un `reset --hard HEAD~2` **volontaire**, puis récupérez les commits « perdus » grâce à `git reflog`.
5. **Stash.** Commencez une modification, basculez d'urgence sur `main` via `git stash`, revenez sur votre branche et restaurez avec `git stash pop`.

Solutions des exercices

Solution 1 — Amend

```
echo "v1" > fichier.txt
git add . && git commit -m "Ajoute fchier"      # faute : « fchier »

git commit --amend -m "Ajoute le fichier"      # remplace le dernier commit
git log --oneline -1                          # le message est corrigé, un seul
commit
```

Le hash a changé : `--amend` n'édite pas le commit, il en fabrique un nouveau à sa place. Sans risque ici car il n'a pas été poussé.

Solution 2 — Rebase interactif

```
git switch -c nettoyage
echo a >> f.txt && git add . && git commit -m "wip"
echo b >> f.txt && git add . && git commit -m "wip encore"
echo c >> f.txt && git add . && git commit -m "fix"
echo d >> f.txt && git add . && git commit -m "ça marche"
```

```
git rebase -i HEAD~4
```

Dans l'éditeur, gardez `pick` sur la **première** ligne, mettez `fixup` (ou `squash`) sur les **trois** suivantes :

```
pick 1111111 wip
fixup 2222222 wip encore
fixup 3333333 fix
fixup 4444444 ça marche
```

Sauvegardez, fermez. Résultat :

```
git log --oneline
# * a1b2c3d wip      ← UN seul commit (renommez-le avec reword si vous voulez un
vrai message)
```

Solution 3 — Reset vs revert

```
git switch -c test-annulation
echo un > x.txt && git add . && git commit -m "Commit A"
echo deux > x.txt && git add . && git commit -m "Commit B"
echo trois > x.txt && git add . && git commit -m "Commit C"
```

```
# Annuler C avec reset : il DISPARAÎT de l'historique
git reset --hard HEAD~1
git log --oneline
# * ... Commit B      ← C n'est plus là du tout
```

```
# Annuler B avec revert : un NOUVEAU commit apparaît, B reste visible
git revert HEAD --no-edit
git log --oneline
# * ... Revert "Commit B"  ← ajouté
# * ... Commit B          ← toujours présent dans l'historique
```

La différence est nette : reset efface (remonte le temps, local uniquement), revert ajoute un contre-commit (avance le temps, sûr partout).

Solution 4 — Sauvetage au reflog

```
git switch -c test-sauvetage
echo 1 > s.txt && git add . && git commit -m "Précieux 1"
echo 2 > s.txt && git add . && git commit -m "Précieux 2"

git reset --hard HEAD~2      # « catastrophe » : les deux commits semblent
perdus
git log --oneline            # ils n'apparaissent plus

git reflog
# a1b2c3d HEAD@{1}: commit: Précieux 2  ← les voilà, toujours là
# ...
git reset --hard HEAD@{1}    # on revient à « Précieux 2 »
git log --oneline            # les deux commits sont récupérés
```

Rien n'était perdu : reset --hard avait seulement déplacé l'étiquette, les commits restaient accrochés au reflog.

Solution 5 — Stash

```
git switch feature/en-cours
echo "modif à moitié faite" >> travail.txt  # PAS committé, on n'est pas prêt

git stash      # range la modif, nettoie le répertoire
git status     # « working tree clean » : travail.txt est
revenu à l'état committé

git switch main      # on peut basculer sans rien committer de bâclé
# ... on règle l'urgence sur main ...

git switch feature/en-cours
git stash pop        # la modif à moitié faite revient, prête à
continuer
git status           # travail.txt est de nouveau modifié
```

Points clés à retenir

- **Règle d'or** : ne jamais réécrire un historique **partagé** (amend, rebase, reset sont pour le local) ; sur main, on annule avec revert.

- `rebase -i` avant d'ouvrir une PR = un historique propre, agréable à relire.
- `reset` **remonte le temps** (déplace la branche, local) ; revert **ajoute un contre-commit** (sûr partout).
- `reset --hard` détruit le travail non committé — vérifiez `git status` **avant**.
- `git reflog` rattrape presque toutes les erreurs : tant qu'il tient, rien n'est vraiment perdu.